SATURN: Splitting Approach on Tensorized Ubiquitous Regression Networks

Anonymous Author(s)
Affiliation
Address
email

Abstract

Convolutional Neural Networks often contain fully connected layers after their convolution phase, which requires the data to be flattened. Despite empirical success, this flattening ruins the multidimensional structure of the input data and increases the number of parameters, which may cause overfitting. Given the tensor nature of the input data, fully connected layers can be replaced with tensorized low-rank layers, which, besides considering the nature of the data, reduce the number of parameters. However, these layers perform poorly when the approximated rank is too low. To address this issue and further reduce the number of parameters while preserving the quality of the network, we propose a framework that, by using two splitting methods, transforms the multidimensional tensor into a thinner and higher-order tensor and applies tensorized layers on this high-order tensor. This leads to fewer parameters and less computational cost while preserving the quality of the network. In our experiments, evaluated on CIFAR10, MNIST, and Tiny ImageNet-200, we have successfully reduced the classifier parameters of well-known models such as VGG19, ResNet50, and ResNet101 by more than 95%, while also achieving better accuracy and overcoming overfitting. Compared to traditional state-of-the-art models, our approach achieves significant parameter reduction and compression while preserving accuracy.

1 Introduction

Neural Networks, especially those that are applied to images such as *Convolutional Neural Network* (CNNs), often consist of two main parts: One that is responsible to extract features from the input data, and the other part is a *MultiLayer Perceptron* (MLP) which Maps the extracted features to the desired space. Despite the importance of data and their spatial relationship, maintaining their topological structure is often discarded since many known CNNs like ResNet or VGG, seem to work fine; However in cases where these structures are more important than the others such as MRI, It is crucial to adapt the known networks to work without destroying the multi-linear structure of the data. While preserving the natural structure of the data throughout the network is important, a greater drawback from using MLPs is their excessive required number of parameters which not only limit their capabilities in real time and practical problems but also prone them to overfitting.

Most natural datasets can be represented as a Tensor due their multi-linear or multi-modal structure; Images can be considered as a 3-order tensors with width, height and the color channels as their corresponding 3 modes, the same could be considered for videos as 4-order tensor with an extra time mode. In practice, most of the used data in the current machine learning and artificial intelligence tasks can be encoded as tensors, requiring their expansion to the linear algebra.

In machine learning, we usually consider data as vectors and datasets will form a set of vectors (ideally called a matrix). By that definition, many matrix decomposition methods have been invented

Submitted to 38th Conference on Neural Information Processing Systems (NeurIPS 2024). Do not distribute.

to work on such matrices. Tensor methods are to generalize these methods to a higher dimensional space. Mathematically, tensors, have been widely studied in the past. One excellent example within tensor methods is Tensor Decomposition which can be considered as a prime backbone for most tensorized layers and approximations in deep learning [1] [2], Signal Processing [3] [4], fusion [5], blind source seperation [4] [6], and computer vision [7] .

Considering the new advances in *Deep Neural Networks* (DNNs) and their applications to computer vision such as image recognition, image segmentation, image generation, video generation, vision transformers, etc ...; most architectures still flatten the data before feeding them to their MLPs which not only destroys the natural structure of the data but also increase the number of parameters needed for the model.

Images as the frequent example of data used in machine learning tasks are considered 3-order tensors which require their own algebra to be applied when fed to the MLP. *Fully Connected* (FC) layers used in many architectures are the main culprit behind vectorizing the data and the number of parameters. To alleviate this situation, a better replacement for FC layers is needed which not only preserves the current model's accuracy but also reduces the number of required parameters and does not destroy the structure of data.

One possible working solution to tensorizing FC layers is by employing the low rank approximation of the weight tensor. Some works have been done to address these issues which will be discussed in Section 2 . However, those methods usually perform poorly when the approximated rank is too low which is not ideal for larger models. The cost of these models usually depend on the size of the tensor, our experiment shows that a thinner while higher dimensional reshaped tensor of the original input tensor, can perform much better in lower ranks and by a unique splitting and stacking technique, we preserved the models' accuracy. In this paper, we have provided two splitting approaches which have their own use dependent on the model's task; Using these methods, we have reshaped the tensor to a higher dimensional space where the cost of computation is less and has a better capability to handle lower approximated ranks. Our methods evaluated on CIFAR10, MNIST, and Tiny-ImageNet-200 applied to known CNN models such as ResNet50, ResNet101, and VGG19, as well as a typical CNN achieved similar accuracy in most methods with more than 95 percent less parameters compared to the FC layers and even overcame overfitting in some cases.

2 Related works

Several prior papers address the importance of using tensors and preserving the natural structure, One uses CP decomposition to enhance convolution layers [8]. In [9], tucker decomposition is used for convolution enhancement with fewer parameters. A weight sharing scheme in multi tasking learning is proposed by [10]. In [11], a Tensor-Train decomposition is used as a low rank approximation of the weight tensor. Some uses tensor methods for model compression and acceleration [12].

In general, tensor methods can be widely used in many machine learning and deep learning tasks, especially computer vision [13]. Their use, can be categorized into three main approaches: The first approach compresses the tuned parameters of the pre-trained network while preserving the quality of network with these approximated parameters. So here the new network is not trained and the approximated network could only be used in the test step. Two methods for approximation of the tuned parameters of pertained CNN in each layer based on matrix and tensor decompositions adjusted with clustering of inlet and outlet layers have been proposed [14]. An improvement is reported in [15]. Tensor CP and Tucker decompositions are used to approximate the kernels of pre-trained CNN networks [16] [17] [18]. In [18] a tensor based method to find an approximation of fully connected layer parameters in CNN networks is introduced. Despite the proper implementation results, these methods have the following drawbacks: increase cost of pre-trained networks and finding these compressions is also costly. It also cannot be used in applications such as deep reinforcement learning where the network constantly should be learned. On the other hand, this approach has confined itself to compressing convolutional methods for the test phase, although a deeper question can be asked. Why should we compress an already trained network and not seek for a compact network representation that can be trained from scratch?

The second approach consider the convolutional layers as a base, and approximate or represent them with smaller parameters, to design end-to-end trainable neural network. Different kinds of methods

have been proposed to design end-end learnable networks based on approximation of convolution process with different views: For example, in [19] and [20] , the rank-1 approximation of 3-order tensor filters corresponding to each outlet channel is used. In [21] ,a share factors corresponding to inlet and outlet are considered to reduce the redundancy of parameters. Recently, in [22] by considering the 3-order filters of each outlet channel as rank-k tensor it has been shown that the pixels of each outlet channel is the contraction of this tensor with the corresponding patch of all input channels. Unlike standard convolution, Mobile Nets proposed in [23] separate the spatial filtering and integration steps. For each input channel, only a single spatial filtering is performed, and by different combination of these filtered images, different output channels are obtained.

The third approach do not consider Convolutional structure as a base and try to find other type of layers to work on multidimensional data. One notable recent work is Tensor Regression Networks [1]. They used a *Tensor Regression Layer* (TRL) and *Tensor Contraction Layer* (TCL) to preserve the structure of the data as they are moving across the network while reducing the number of parameters. Their method surprisingly achieved the near similar accuracy as its FC network with more than 65 percent space saving. TRL and TCL will be discussed in section 3, and they are used as the base of our method. Unfortunately, one of the biggest issues with TCL and TRL is their inadequate implementations. They are theoretically required less parameters than their FC versions; However they often perform poorly in terms of the actual execution time. This is one of the biggest limitation on working with tensors since a practical implementation of tensor methods are yet to be made.

The power of tensor regression has been addressed in many prior papers [**?**][**?**] [24] [25]. However, most of these works, usually rely on analytical solution and large tensors. In [26], a non-linear Tensor-Train decomposition is used instead of tucker decomposition. While many works focuses on individual layers, in [27], T-Nets attempts to provide a single high-order, low-rank tensor that jointly capture the full structure of a neural network.

DNNS have been used in many applications and problems, however, many questions still remains in terms of their credibility and why they work so well. DNNs usually contain too many parameters and an important question is whether they require that many parameters. By tensorizing the DNNs, we can address these issues and attempt to better understand the concepts of deep learning. For example, one uses tensor methods as a tool to study CNNs [28], they follow up by studying the power of overlapping architectures in deep learning [29]. Some provide global optimal and optimization for non-convex factorization problems [30]. In [31], they used contraction as a composition operator in *Natural Language Processing* (NLP). Some used tensor methods as a tool to guarantee convergence and consistent learning[32].

Most of the recent works, rely on low rank approximation of the tensor or some tensor decomposition methods as a dimension reduction layer. Despite their success, a fully tensorized end-to-end trainable model requires almost near similar ranks to perform well when compared to their FC competitives. The draw back of using same ranks as the input tensor is the cost. Our method provide a reshaping approach that maintains the accuracy by converting the input tensor into a thinner and higher order tensor that requires fewer parameters and can perform better in lower ranks. To our knowledge, no prior work increases the dimensions of the tensor in order to achieve near similar accuracy while using tensorized layers with lower ranks and fewer parameters.

3 Mathematical background

This paper, requires basic understanding of tensor methods, If readers are already knowledgeable in the matter, they can skip to section 4.

3.1 Notations

For a 3-order tensor $\mathcal{T} \in \mathbb{R}^{I_1 \times I_2 \times I_3}$, there are 3 modes with respected sizes I_1, I_2, and I_3. Its $\mathcal{T}_{i_1,i_2,i_3}$ **element** is a 0-order tensor (scalar), (i_1, i_2, i_3). **Slices** of its first and second modes are 2-order tensors (matrices) $\mathbf{M}$ and are denoted by two colons, $\mathcal{T}_{:,:,i_3}$. **Fibers** of its first mode are 1-order tensors (vectors) $\mathbf{V}$ and are denoted by a colon, $\mathcal{T}_{:,i_2,i_3}$.

3.2 Tensor folding and unfolding

Tensor folding and unfolding are fundamental operations in tensor analysis, allowing for the manipulation and transformation of tensors into different dimensional structures. Tensor unfolding, also known as matricization, converts a higher-order tensor into a matrix or a vector. Let $\mathcal{T} \in \mathbb{R}^{I_1 \times I_2 \times \ldots \times I_N}$ be an N-order tensor. In our case, tensor folding folds an unfolded tensor back.

Mode-n unfolding : Unfolding $\mathcal{T}$ along mode-n results in a matrix representation of mode-n. This operation is denoted as $T_{[n]}$. The unfolding is achieved by rearranging the entries of $\mathcal{T}$ into a matrix such that the mode-n indices become the rows of the matrix. The mapping from tensor indices to matrix indices is given by:

$$\begin{aligned} &(i_1, \ldots, i_n, \ldots, i_N) \longrightarrow (i_n, r) \\ &\text{where } r = \sum_{\substack{p=1 \\ p \neq n}}^{N} i_p \times \prod_{\substack{l=p+1 \\ l \neq n}}^{N} I_l \end{aligned} \tag{1}$$

Each row of the resulting matrix corresponds to a different combination of indices for the remaining modes.

Vectorization : Similar to mode-n unfolding, in tensor vectorization we map each element within the tensor to a element j of $Vec(\mathcal{T})$:

$$\begin{aligned} &(i_1, \ldots, i_n, \ldots, i_N) \longrightarrow (r) \\ &\text{where } r = \sum_{p=1}^{N} i_p \times \prod_{l=p+1}^{N} I_l \end{aligned} \tag{2}$$

3.3 Tensor contraction

Tensor contraction is a fundamental operation that allows for the multiplication and simplification of tensors and multiplies elements of two tensors and provides a way to combine tensors by summing over shared indices.

Tensor n-mode product : Tensor n-mode product, is a tensor operation that allows for the multiplication of a tensor with a matrix along a specific mode. Given a tensor $\mathcal{T} \in \mathbb{R}^{I_1 \times I_2 \times \cdots \times I_N}$ and a matrix $\mathbf{M} \in \mathbb{R}^{J \times I_n}$, the n-mode product of $\mathcal{T}$ and $\mathbf{M}$, denoted as $\mathcal{T} \times_n \mathbf{M}$, results in a tensor of size $\mathbb{R}^{I_1 \times \cdots \times I_{n-1} \times J \times I_{n+1} \times \cdots \times I_N}$. The n-mode product can be expressed mathematically as:

$$(\mathcal{T} \times_n \mathbf{M})_{i_1, \ldots, i_{n-1}, j, i_{n+1}, \ldots, i_N} = \sum_{i_n=0}^{I_n} \mathcal{T}_{i_1, \ldots, i_n, \ldots, i_N} \mathbf{M}_{j, i_n} \tag{3}$$

where $i_1, \ldots, i_{n-1}, j, i_{n+1}, \ldots, i_N$ are the indices of the resulting tensor, and i_n is the summation index. Alternatively, the n-mode product can be expressed using the tensor unfolding operation along the n-th mode, denoted as $\mathbf{T}_{(n)}$. In this case, the n-mode product can be written as:

$$\mathcal{T} \times_n \mathbf{M} = \mathbf{M}\mathbf{T}_{[n]} \tag{4}$$

Generalized inner product : For two tensors $\mathcal{X} \in \mathbb{R}^{I_x \times I_1 \times I_2 \times \cdots \times I_N}$ and $\mathcal{Y} \in \mathbb{R}^{I_1 \times I_2 \times \cdots \times I_N \times I_y}$ sharing N modes of the same size, a generalized inner product along the N last modes of X and Y is defined as:

$$\langle \mathcal{X}, \mathcal{Y} \rangle_N = \sum_{i_1=0}^{I_1 - 1} \cdots \sum_{i_N=0}^{I_N - 1} \mathcal{X}_{:, i_1, i_2, \ldots, i_N} \mathcal{Y}_{i_1, i_2, \ldots, i_N, :} \tag{5}$$

where $\langle \mathcal{X}, \mathcal{Y} \rangle_N \in \mathbb{R}^{I_x \times I_y}$.

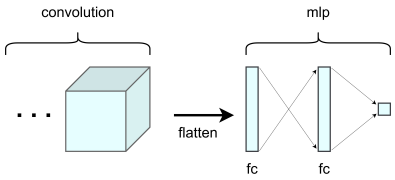


Figure 1: A simple MLP after some convolution layers.

3.4 Tucker decomposition

The Tucker decomposition is a powerful method in tensor algebra that breaks down a tensor into a core tensor multiplied by matrices along each mode. In the Tucker decomposition, a tensor $\mathcal{T} \in \mathbb{R}^{I_1 \times I_2 \times \cdots \times I_N}$ is represented as the product of a core tensor $\mathcal{G} \in \mathbb{R}^{J_1 \times J_2 \times \cdots \times J_N}$ and a set of factor matrices $\mathbf{A}^{(1)} \in \mathbb{R}^{I_1 \times J_1}, \mathbf{A}^{(2)} \in \mathbb{R}^{I_2 \times J_2}, \ldots, \mathbf{A}^{(N)} \in \mathbb{R}^{I_N \times J_N}$ along each mode. Mathematically, the Tucker decomposition can be expressed as:

$$\mathcal{T} = \mathcal{G} \times_1 \mathbf{A}^{(1)} \times_2 \mathbf{A}^{(2)} \times \cdots \times_N \mathbf{A}^{(N)} \tag{6}$$

where $\times_n$ denotes the n-mode product. The Tucker decomposition can be computed using the mode-n unfolding of the tensor, denoted as $\mathbf{T}_{[n]}$. By reshaping the tensor along each mode, the Tucker decomposition can be written as:

$$\mathbf{T}[n] = \mathbf{A}^{(n)} \mathbf{G}[n] \left(\mathbf{A}^{(1)} \otimes \cdots \otimes \mathbf{A}^{(n-1)} \otimes \mathbf{A}^{(n+1)} \otimes \cdots \otimes \mathbf{A}^{(N)} \right)^T \tag{7}$$

where $\mathbf{G}_{[n]}$ represents the unfolding of the core tensor along mode n.

3.5 Tensor regression networks

The combination of tensor regression and tensor contractions has been used in an end-to-end trainable model by Kossaifi [1], as discussed in section 2 Given a MLP, there are two ways to interpret FC layers: One is to consider them as a feature extraction layer where they map a latent or feather space to another latent space, on another hand, we can consider them as regression layers which are basically the classifier layers and given the purpose of the network, they can be used interchangeably. In figure 1 a very simple MLP that is being used after some convolution layers is provided. In this case, the first FC layer can be considered a dimension reduction layer while the second FC layer maps the feature space to the target space. In the following text, we will discuss to main approaches to replace such FC layers:

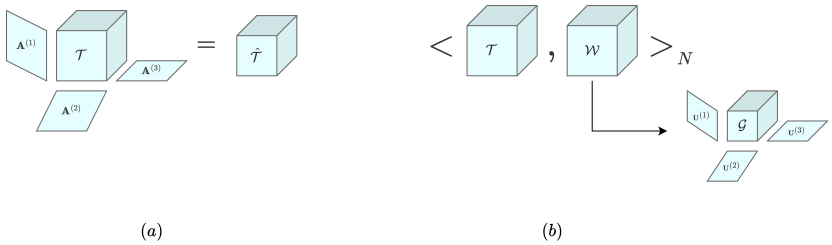
3.5.1 Tensor contraction layer

Tensor Contraction Layer (TCL) is a form of feature extraction and could also be used for embedding. The prime idea behind TCL is Tucker decomposition; Via multiplying matrices to each mode of the input tensor, we can obtain a denser tensor (dependent of the shape of the matrices). Figure 2 demonstrate the TCL process on a 3-order tensor.

Considering an input tensor $\mathcal{X}$ of size $(S_0, I_1, \ldots, I_N)$ and a size $(R_1, \ldots, R_N)$ TCL, where S_0 is the size of the batch and (I_i, R_i) is the shape of the matrix applied to mode i of the tensor; The number of required parameters is calculated as :

$$\sum_{i=1}^{N} R_i \times I_i \tag{8}$$

And the resulting tensor is of size $(S_0, R_1, \ldots, R_N)$. Compared to the FC version where the simple regression between these two tensors would require at least $\prod_{i=1}^{N} I_i \times R_i$ parameters.



(a) (b)

Figure 2: Tensor regression networks, replacement for FC layers. In (a), A simple TCL is applied; Each mode i of tensor $\mathcal{T}$ is multiplied by factor $\mathbf{A}^{(i)}$. In (b), A TRL is applied; Generalized inner product of input tensor $\mathcal{T}$ and weight tensor $\mathcal{W}$ is calculated as the output tensor while considering a low rank approximation of weight tensor $\mathcal{W}$ in a tucker decomposed form.

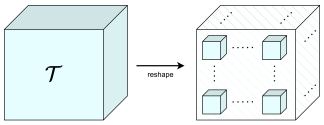


Figure 3: An outline of reshaping to higher orders tensor.

3.5.2 TRL

Tensor Regression Layers (TRL) is a replacement for the FC regression layers. A simple regression that is the foundation of a MLP is a trivial generalized inner product of $\mathcal{W}$ (Weight Tensor) with the $\mathcal{X}$ (Input tensor). The idea behind TRL is to reconstruct W from a low rank approximation of itself via tucker decomposition. In figure 2, a TRL application in a MLP is provided. For the sake of simplicity, we considered a binary classification problem where the output of the inner product is a scaler. Similarly, if the desired output is a vector or even a tensor, $\mathcal{W}$ will be a higher dimensional tensor itself to generate each mode of the output.

Considering an input tensor $\mathcal{X}$ of size $(S_0, I_1, \ldots, I_N)$, an output tensor $\mathcal{Y}$ of size $S_0 \times O$, and a size $(R_1, \ldots, R_N, R_{N+1})$ TRL with bias b of size O such that $\mathcal{Y} = \langle X, W \rangle_N + b$; The number of required parameters can be calculated as :

$$\prod_{i=1}^{N+1} R_i + \left(\sum_{i=1}^{N} R_i \times I_i + R_{N+1} \times O \right) \tag{9}$$

Compared to the FC version where the number of parameters is $O \times \prod_{i=1}^{N} I_i$.

4 Splitting approach

Impressed by the vision transformers patching and down sampling methods, we can transform our input tensor into a thinner and higher dimensional tensor that not only reduces the number of parameters for a TRL or TCL layer but also maintain the accuracy. However, not any reshaping scheme to arbitrary higher orders and dimensions guarantee that the multi-linear structure is preserved and the accuracy is maintained With a proper splitting scheme, the newly made tenser has better physical interpretation than the flattened data. We introduced two splitting schemes, both of which are useful and could be used differently according to the network architect. Figure 3 demonstrate the general splitting outline. Given a 3-order tensor, if we split across each mode, the output is a 6-order tensor where the first 3 modes correspond the split index, and the next 3 modes are the data in each split.

4.1 Attached Splitting

In attached splitting, every split is continues, that is if mode I_i of the input tensor $\mathcal{X}$ with length n is to be split into 2 parts, then the split indices are $(0, \frac{n}{2})$ and $(\frac{n}{2}, n)$. Figure 4 demonstrate the splitting process on a matrix.

Given a tensor $\mathcal{T} \in \mathbb{R}^{I_1 \times \cdots \times I_N}$ and split sizes $(L_1, \ldots, L_N)$, attached mapping of elements is done via:

$$\mathcal{T}[i_1, \ldots, i_N] \longrightarrow \mathcal{T}_a \left[\lfloor \frac{L_1 \times i_1}{I_1} \rfloor, \ldots, \lfloor \frac{L_N \times i_N}{I_N} \rfloor, i_1 \bmod \frac{I_1}{L_1}, \ldots, i_N \bmod \frac{I_N}{L_N}, \right] \tag{10}$$

where $\mathcal{T}_a \in \mathbb{R}^{L_1 \times \cdots \times L_N \times \frac{I_1}{L_1} \times \cdots \times \frac{I_N}{L_N}}$.

4.2 Compressed Splitting

In compressed splitting, we incorporate the idea behind down sampling, that is if mode I_i of input tensor $\mathcal{X}$ with length n is to be split into 2 part , different to the attached splitting, the indices are even-odd combinations; That is $\{t | t \in (0, n) \wedge (t \bmod 2 = 0)\}$ and $\{t | t \in (0, n) \wedge (t \bmod 2 = 1)\}$.

Given a tensor $\mathcal{T} \in \mathbb{R}^{I_1 \times \cdots \times I_N}$ and split sizes $(L_1, \ldots, L_N)$, compressed mapping of elements is done via:

$$\mathcal{T}[i_1, \ldots, i_N] \longrightarrow \mathcal{T}_c \left[i_1 \bmod L_1, \ldots, i_N \bmod L_N, \lfloor \frac{i_1}{L_1} \rfloor, \ldots, \lfloor \frac{i_N}{L_N} \rfloor, \right] \tag{11}$$

where $\mathcal{T}_c \in \mathbb{R}^{L_1 \times \cdots \times L_N \times \frac{I_1}{L_1} \times \cdots \times \frac{I_N}{L_N}}$.

4.3 Method Analysis

Considering an input tensor $\mathcal{X}$ of size $(S_0, I_1, \ldots, I_N)$, either reshaping would result in the tensor $\hat{\mathcal{X}}$ of size $(S_0, L_1, \ldots, L_N, \frac{I_1}{L_1}, \ldots, \frac{I_N}{L_N})$. An example of each reshaping approach is done in figure 5 on a sample image.

In case of a $(r_1 \ldots, r_N, R_i, \ldots, R_N)$ TCL, Since L_is are usually small, the number of parameters required for them $\sum_{i=0}^{N} L_i \times r_i$ is also small; Similarly, $\frac{I_i}{L_i}$ is smaller than I_i so the new desired R_is is smaller than the previous R_is and the total number of parameters can be calculated as:

$$\sum_{i=1}^{N} L_i \times r_i + \sum_{i=1}^{N} R_i \times \frac{I_i}{L_i} \tag{12}$$

which is ideally less than equation 8.

In case of $(r_1, \ldots, r_N, R_1, \ldots, R_N, R_{N+1})$ TRL, the same logic as TCL could also be applied. The total number of parameters in a new TRL is calculated as:

$$\left(\prod_{i=1}^{N} r_i \prod_{i=1}^{N+1} R_i \right) + \left(\sum_{i=1}^{N} L_i \times r_i + \sum_{i=1}^{N} R_i \times \frac{I_i}{L_i} + R_{N+1} \times O \right) \tag{13}$$

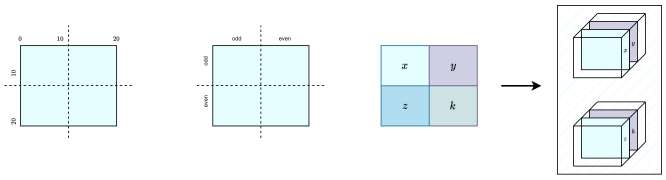
which is ideally less than equation 9.

5 Experiments

6 Conclusion

References

[1] Jean Kossaifi, Zachary C Lipton, Arinbjorn Kolbeinsson, Aran Khanna, Tommaso Furlanello, and Anima Anandkumar. Tensor regression networks. *Journal of Machine Learning Research*, 21(123):1–21, 2020.



(a) (b) (c)

Figure 4: (a) attached splitting of a 2-order tensor. (b) compressed splitting of a 2-order tensor. (c) demonstration of stacking split chunks of a 2-order tensor into a 4-order tensor.

(a) (b) (c)

Figure 5: (a) original sample image. (b) reshaped sample image using attached approach. (c) reshaped sample image using compressed approach.

[2] Tamara G Kolda and Brett W Bader. Tensor decompositions and applications. *SIAM review*, 51(3):455–500, 2009.

[3] Nicholas D Sidiropoulos, Lieven De Lathauwer, Xiao Fu, Kejun Huang, Evangelos E Papalexakis, and Christos Faloutsos. Tensor decomposition for signal processing and machine learning. *IEEE Transactions on signal processing*, 65(13):3551–3582, 2017.

[4] RZdunek ACichocki et al. Nonnegativematrixandtensor factorizations: Applicationstoexploratorymulti-waydata analysisandblindsourceseparation, 2009.

[5] Evangelos E Papalexakis, Christos Faloutsos, and Nicholas D Sidiropoulos. Tensors for data mining and data fusion: Models, applications, and scalable algorithms. *ACM Transactions on Intelligent Systems and Technology (TIST)*, 8(2):1–44, 2016.

[6] Martijn Bousse, Otto Debals, and Lieven De Lathauwer. A tensor-based method for large-scale blind source separation using segmentation. *IEEE Transactions on Signal Processing*, 65(2):346–358, 2016.

[7] M Alex O Vasilescu and Demetri Terzopoulos. Multilinear analysis of image ensembles: Tensorfaces. In *Computer VisionECCV 2002: 7th European Conference on Computer Vision Copenhagen, Denmark, May 28–31, 2002 Proceedings, Part I 7*, pages 447–460. Springer, 2002.

[8] Vadim Lebedev, Yaroslav Ganin, Maksim Rakhuba, Ivan Oseledets, and Victor Lempitsky. Speeding-up convolutional neural networks using fine-tuned cp-decomposition. *arXiv preprint arXiv:1412.6553*, 2014.

[9] Cheng Tai, Tong Xiao, and Xiaogang Wang. Weinan e. *Convolutional neural networks with low-rank regularization. CoRR, abs/1511.06067*, 3:6, 2015.

[10] Yongxin Yang and Timothy Hospedales. Deep multi-task representation learning: A tensor factorisation approach. *arXiv preprint arXiv:1605.06391*, 2016.

[11] Alexander Novikov, Dmitrii Podoprikhin, Anton Osokin, and Dmitry P Vetrov. Tensorizing neural networks. *Advances in neural information processing systems*, 28, 2015.

[12] Lei Deng, Guoqi Li, Song Han, Luping Shi, and Yuan Xie. Model compression and hardware acceleration for neural networks: A comprehensive survey. *Proceedings of the IEEE*, 108(4):485–532, 2020.

[13] Yannis Panagakis, Jean Kossaifi, Grigorios G Chrysos, James Oldfield, Mihalis A Nicolaou, Anima Anandkumar, and Stefanos Zafeiriou. Tensor methods in computer vision and deep learning. *Proceedings of the IEEE*, 109(5):863–890, 2021.

[14] Emily L Denton, Wojciech Zaremba, Joan Bruna, Yann LeCun, and Rob Fergus. Exploiting linear structure within convolutional networks for efficient evaluation. *Advances in neural information processing systems*, 27, 2014.

[15] Max Jaderberg, Andrea Vedaldi, and Andrew Zisserman. Speeding up convolutional neural networks with low rank expansions. *arXiv preprint arXiv:1405.3866*, 2014.

[16] Vadim Lebedev, Yaroslav Ganin, Maksim Rakhuba, Ivan Oseledets, and Victor Lempitsky. Speeding-up convolutional neural networks using fine-tuned cp-decomposition. *arXiv preprint arXiv:1412.6553*, 2014.

[17] Marcella Astrid and Seung-Ik Lee. Cp-decomposition with tensor power method for convolutional neural networks compression. In *2017 IEEE International Conference on Big Data and Smart Computing (BigComp)*, pages 115–118. IEEE, 2017.

[18] Yong-Deok Kim, Eunhyeok Park, Sungjoo Yoo, Taelim Choi, Lu Yang, and Dongjun Shin. Compression of deep convolutional neural networks for fast and low power mobile applications. *arXiv preprint arXiv:1511.06530*, 2015.

[19] Jonghoon Jin, Aysegul Dundar, and Eugenio Culurciello. Flattened convolutional neural networks for feedforward acceleration. *arXiv preprint arXiv:1412.5474*, 2014.

[20] Min Wang, Baoyuan Liu, and Hassan Foroosh. Factorized convolutional neural networks. In *Proceedings of the IEEE international conference on computer vision workshops*, pages 545–553, 2017.

[21] Cheng Tai, Tong Xiao, Yi Zhang, Xiaogang Wang, et al. Convolutional neural networks with low-rank regularization. *arXiv preprint arXiv:1511.06067*, 2015.

[22] Dat Thanh Tran, Alexandros Iosifidis, and Moncef Gabbouj. Improving efficiency in convolutional neural networks with multilinear filters. *Neural Networks*, 105:328–339, 2018.

[23] Andrew G Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. Mobilenets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861*, 2017.

[24] Hua Zhou, Lexin Li, and Hongtu Zhu. Tensor regression with applications in neuroimaging data analysis. *Journal of the American Statistical Association*, 108(502):540–552, 2013.

[25] Rose Yu and Yan Liu. Learning from multiway data: Simple and efficient tensor regression. In *International Conference on Machine Learning*, pages 373–381. PMLR, 2016.

[26] Dingheng Wang, Guangshe Zhao, Hengnu Chen, Zhexian Liu, Lei Deng, and Guoqi Li. Nonlinear tensor train format for deep neural network compression. *Neural Networks*, 144:320–333, 2021.

[27] Jean Kossaifi, Adrian Bulat, Georgios Tzimiropoulos, and Maja Pantic. T-net: Parametrizing fully convolutional nets with a single high-order tensor. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 7822–7831, 2019.

[28] Nadav Cohen, Or Sharir, and Amnon Shashua. On the expressive power of deep learning: A tensor analysis. In *Conference on learning theory*, pages 698–728. PMLR, 2016.

[29] Or Sharir and Amnon Shashua. On the expressive power of overlapping architectures of deep learning. *arXiv preprint arXiv:1703.02065*, 2017.

[30] Benjamin D Haeffele and René Vidal. Global optimality in tensor factorization, deep learning, and beyond. *arXiv preprint arXiv:1506.07540*, 2015.

[31] Edward Grefenstette and Mehrnoosh Sadrzadeh. Experimental support for a categorical compositional distributional model of meaning. *arXiv preprint arXiv:1106.4058*, 2011.

[32] Hanie Sedghi and Anima Anandkumar. Training input-output recurrent neural networks through spectral methods. *arXiv preprint arXiv:1603.00954*, 2016.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer NA means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A No or NA answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer NA means that the paper has no limitation while the answer No means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory Assumptions and Proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer NA means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental Result Reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer NA means that the paper does not include experiments.
   - If the paper includes experiments, a No answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
   - If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
   - Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
   - While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
     (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
     (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
     (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
     (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer NA means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (`https://nips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so No is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://nips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental Setting/Details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer, etc.) necessary to understand the results?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer NA means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment Statistical Significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer NA means that the paper does not include experiments.
   - The authors should answer "Yes" if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
   - The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
   - The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
   - The assumptions made should be given (e.g., Normally distributed errors).
   - It should be clear whether the error bar is the standard deviation or the standard error of the mean.

- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g. negative error rates).
- If error bars are reported in tables or plots, The authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments Compute Resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer NA means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code Of Ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer NA means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer No, they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader Impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer NA means that there is no societal impact of the work performed.
    - If the authors answer NA or No, they should explain why their work has no societal impact or why the paper does not address societal impact.
    - Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
    - The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to

generate deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.

- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pretrained language models, image generators, or scraped datasets)?

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer NA means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
    - Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
    - We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer NA means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.
    - The name of the license (e.g., CC-BY 4.0) should be included for each asset.
    - For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
    - If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
    - For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
    - If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New Assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer NA means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and Research with Human Subjects**

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer NA means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional Review Board (IRB) Approvals or Equivalent for Research with Human Subjects**

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer NA means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.